Laboratorio 2 - Agenda de contactos

Informe técnico y evidencia de ejecución

Registro UTF-8: ID 10 bytes; nombre 80; teléfono 25; correo 90; ciudad 50; LF 1. Total: 256 bytes. Cada campo se codifica a UTF-8 y se rellena con espacios mediante bytes.ljust(). Se rechazan valores cuyo ancho exceda su campo.

Acceso aleatorio (main.py, buscar y guardar):

```
archivo.seek(indice[clave] * TAMANO)
self.posicion = archivo.tell()
contacto = decodificar(archivo.read(TAMANO))
archivo.seek(numero * TAMANO)
archivo.write(registro)
```

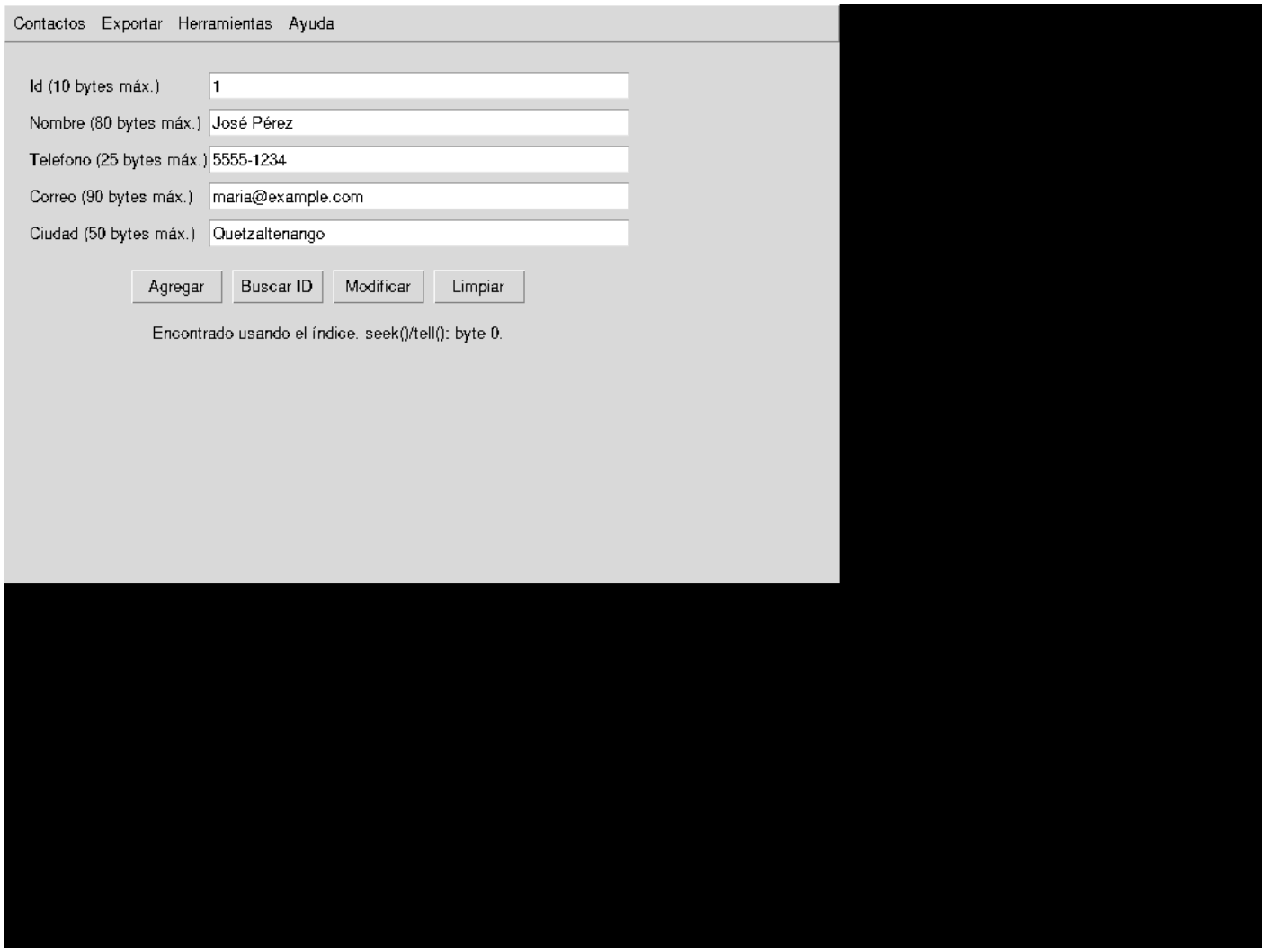
El índice JSON relaciona ID y posición ordinal. Cargarlo cuesta O(n); consultar la clave cargada cuesta O(1) promedio, seguido de una lectura directa de 256 bytes. Una búsqueda secuencial debe recorrer contactos hasta encontrar el ID. El cálculo de SHA-256 sí lee el archivo entero después de guardar.

Integridad: al iniciar se compara el hash recalculado con contactos.sha256. Un cambio externo produce una advertencia y bloquea nuevas escrituras. No se reemplaza automáticamente el hash. Los errores de permisos, ausencia y corrupción se presentan al usuario.

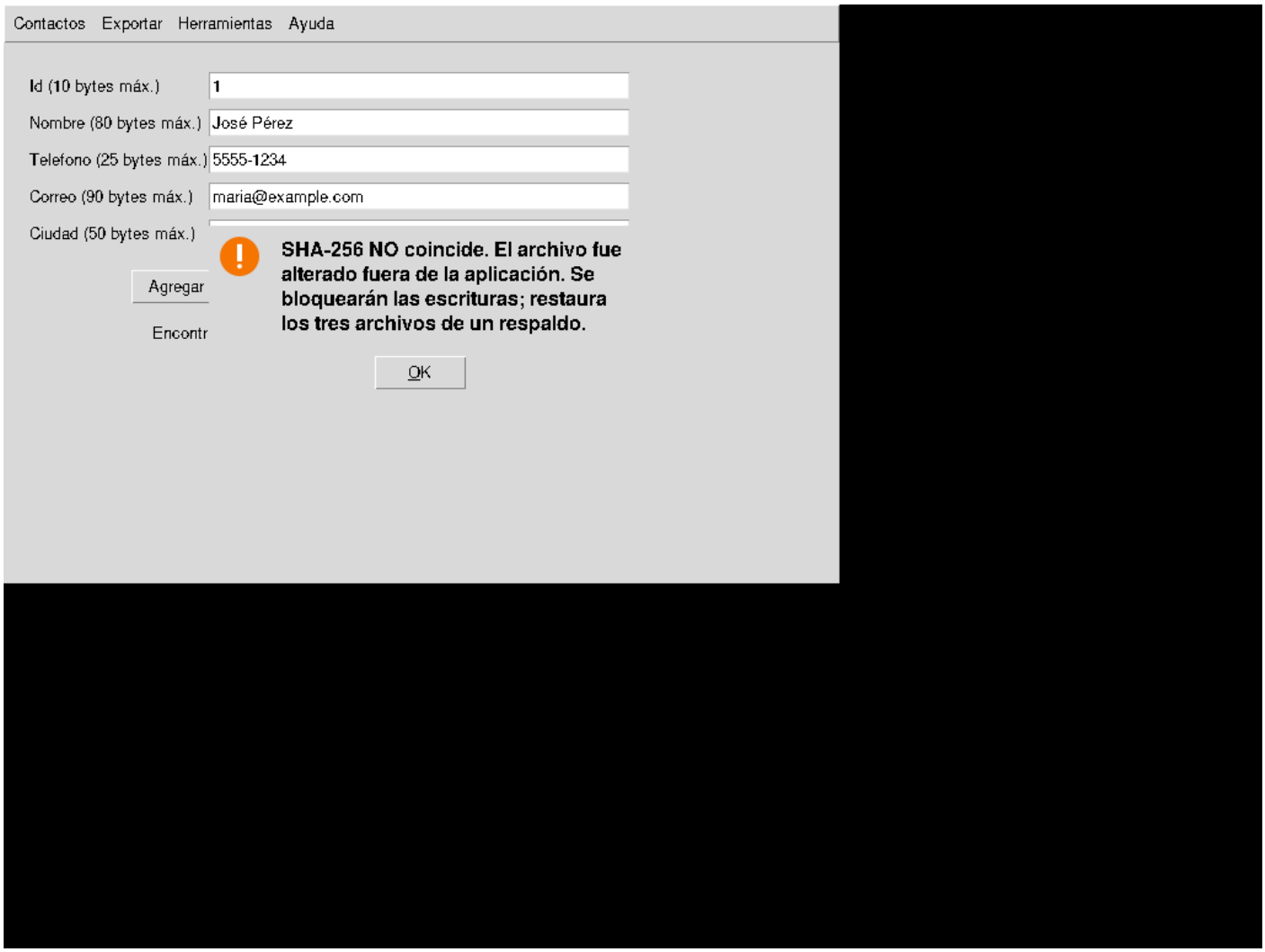
Comparación real: DAT = 768 bytes; DAT + índice + hash = 865 bytes; ZIP completo = 562 bytes. Reducción respecto del conjunto: 35.0%. El ZIP incluye cabeceras y puede crecer para conjuntos muy pequeños.

Validación: cinco pruebas automatizadas aprobadas; modificación conserva los registros vecinos y el tamaño total; tildes UTF-8; ID duplicado/inexistente; corrupción y ausencia del índice; alteración del DAT al reiniciar; exportación JSON/PDF; ZIP íntegro. En pantalla virtual se ejecutaron agregar, buscar y modificar y se verificó el aviso SHA-256.

Interfaz: contacto agregado, buscado y modificado



Evidencia: aviso después de alterar manualmente un byte del DAT



Reproducción: cerrar la aplicación, cambiar una letra en el DAT sin cambiar el tamaño y abrir de nuevo. Restaurar los tres archivos desde un ZIP anterior para continuar. Las capturas usan datos temporales ficticios; los ejemplos entregados conservan su integridad.